TABLE OF CONTENTS

1. Core Architecture & Security Layers 01
2. Identity & Access Management (IAM) 02
3. Passenger Experience (BusGo Client) 03
4. Driver Operations & GPS Streaming 04
5. Conductor & Scanning (BusGo Scanner) 05
6. The Neo Engine: AI & Predictive Intelligence 06
7. Emergency Alert & Incident Response 07
8. Administrative Governance & Analytics 08
9. Database Schema & Data Integrity 09

1. CORE ARCHITECTURE & SECURITY LAYERS

BusGo is built on a high-performance Node.js/Express backend. To maintain "Production-Ready" status, we employ multiple security layers:

- **Helmet.js:** Configures 15 HTTP security headers.
- **CORS:** Strict whitelist-based origin filtering.
- **Rate Limiting:** 10 req/15 min on Auth; 100 req/min general API.
- **Zod Validation:** Every request body is parsed and validated against strict schemas.

2. IDENTITY & ACCESS MANAGEMENT (IAM)

Our security layer utilizes JWT with dual-token rotation.

2.1 MULTI-STEP REGISTRATION

All users must verify their phone number and email. Membership types include standard, premium, and corporate.

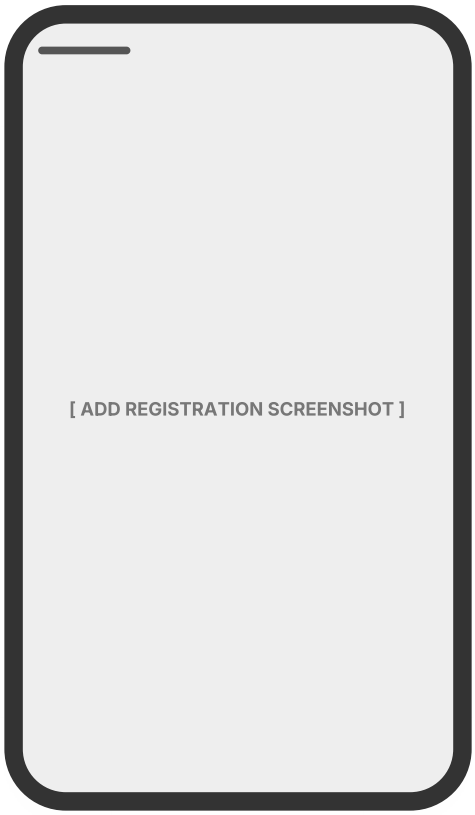

Figure 1: Registration & MFA Interface

3. PASSENGER EXPERIENCE (BUSGO CLIENT)

Designed with Flutter and Provider for elite state management.

3.1 REAL-TIME ROUTE DISCOVERY

Uses Haversine geometry (radius = 0.5km) to identify the closest stops. Subscribes to the bus-locations Supabase broadcast channel for sub-second GPS updates.

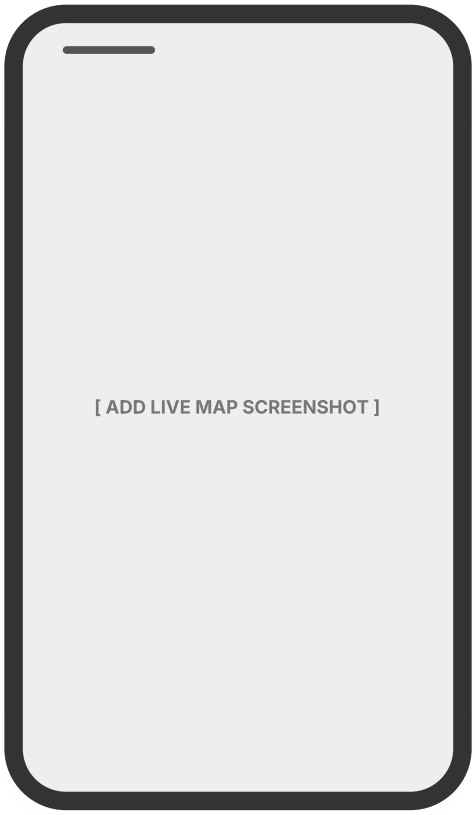

Figure 2: Real-time Bus Tracking Map

4. DRIVER OPERATIONS & GPS STREAMING

Drivers must log in and bind to a specific bus_id. Constant PUT requests to /buses/:id/location stream GPS and heading data.

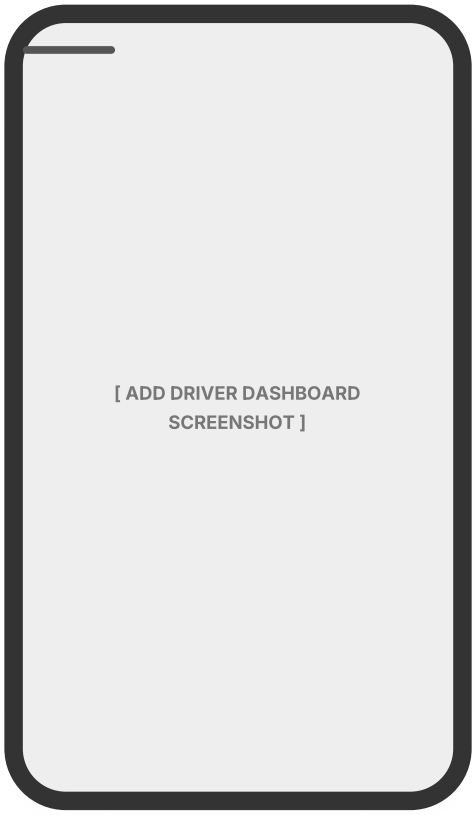

Figure 3: Driver Dashboard & GPS Status

5. CONDUCTOR & SCANNING (BUSGO SCANNER)

Scans the user's qr_token. The backend auto-regenerates tokens if the 30-second TTL expires. Exit Scan triggers trip completion and fare calculation.

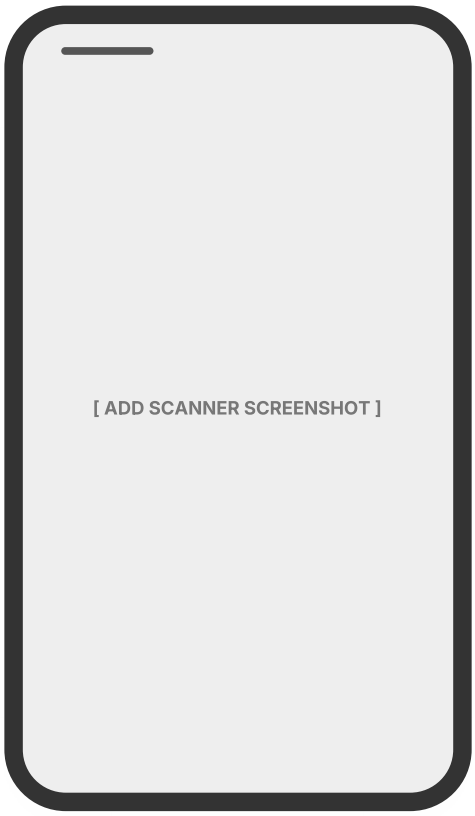

Figure 4: Conductor QR Scanner Interface

6. THE NEO ENGINE: AI & PREDICTIVE INTELLIGENCE

6.1 ETA ESTIMATION PIPELINE

Calculates arrival times using a complex multi-feature model: eta_seconds = (exp(prediction) - 1) * distance_km.

7. EMERGENCY ALERT & INCIDENT RESPONSE

The most critical module for passenger safety. Dispatched to the emergency_alerts table with real-time Admin notification.

Score	Label	Action
5	CRITICAL	Dispatch immediately — call 119/118
4	HIGH	Urgent response required
3	MEDIUM	Monitor and prepare response

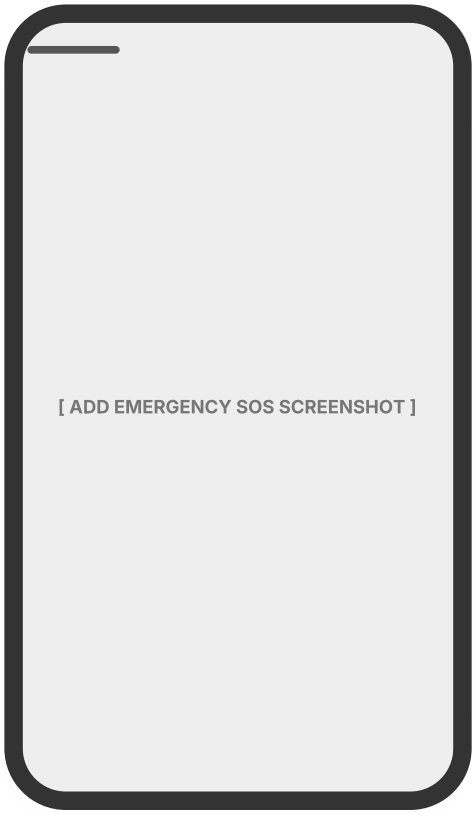

Figure 5: Emergency SOS & Incident Reporting

8. ADMINISTRATIVE GOVERNANCE & ANALYTICS

Full system oversight via the React/TypeScript dashboard. Manage the bus_routes and route_stops tables. Visualize ratings, trips, and revenue via Chart.js.

9. DATABASE SCHEMA & DATA INTEGRITY

BusGo maintains 12 core PostgreSQL tables with strict RLS (Row Level Security).

- **Triggers:** enforce_max_recent_searches() maintains exactly 5 entries per user.
- **Indexes:** 20+ custom indexes ensure sub-100ms response times for geographic queries.